



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

PARADIGMAS DE PROGRAMACIÓN

UNIDAD 4

Dr. Pablo Javier Vidal

Facultad de Ingeniería
Universidad Nacional de Cuyo

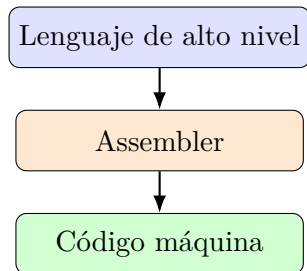
2026

- 1 Ensamblador
- 2 MIPS
- 3 Syscalls y Mensajes
- 4 Ejemplos varios
- 5 Arbol de activación
- 6 Class Instance Record - CIR

Ensamblador

¿Qué es assembler?

- Es un lenguaje de muy bajo nivel, cercano al conjunto de instrucciones de la CPU.
- Cada instrucción corresponde, en general, a una operación elemental del procesador.
- Está fuertemente ligado a una **arquitectura** o **ISA** concreta.
- Permite razonar con precisión sobre:
 - registros, memoria,
 - llamadas, costo de ejecución.



Mnemónicos

En lugar de escribir 011010, escribimos ADD, facilitando la programación directa sobre el hardware.

Características generales de assembler

- **Dependiente de la ISA:** x86, ARM, MIPS, RISC-V, etc.
- **Poca abstracción:** el programador ve registros, direcciones y saltos.
- **Control fino:** útil para estudiar organización de computadoras, calling conventions y acceso a memoria.
- **Legibilidad menor** que en C/Java/Python, pero muy útil para entender cómo trabaja la máquina.
- **No es portable:** un programa MIPS no corre directamente en x86.

Idea central

Assembler permite observar el puente entre el programa fuente y la ejecución real en hardware o en un simulador.

Ventajas y Desventajas

Ventajas

- **Control total:** Acceso directo a registros e interrupciones.
- **Eficiencia:** Código extremadamente rápido y pequeño.
- **Hardware:** Único camino para escribir drivers o BIOS.

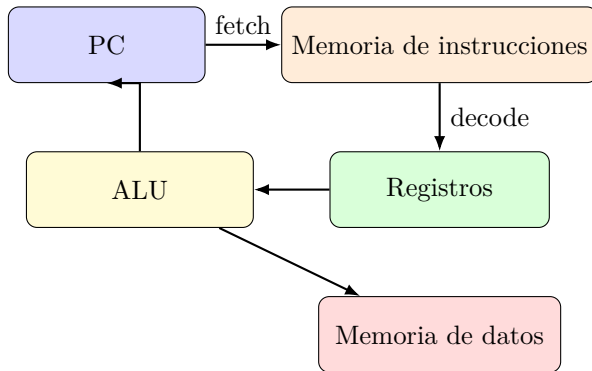
Desventajas

- **Complejidad:** Difícil de leer y mantener.
- **No portable:** Un código x86 no corre en un MIPS.
- **Productividad:** Tareas simples requieren muchas líneas.

Resumen de instrucciones básicas

Categoría	Qué hace	Ejemplos típicos
Movimiento / carga	mueve datos entre memoria y registros	LOAD, STORE, MOV, LW, SW
Aritmética	suma, resta, multiplicación simple	ADD, SUB, ADDI
Lógica	combina bits	AND, OR, XOR
Comparación / test	prepara una decisión	CMP, SLT, BEQ
Control de flujo	cambia el PC	JMP, BNE, JAL, JR
Entrada/salida	servicios del entorno o del simulador	syscall (en MIPS simulados)

¿Cómo trabaja un programa assembler?



- 1 Se lee la instrucción apuntada por el **PC**.
- 2 Se decodifica y se toman operandos de registros o memoria.
- 3 Se ejecuta la operación y se actualiza el PC.

Ejemplo básico en “assembler genérico”

Objetivo en alto nivel

```
int a = 5;  
int b = 7;  
int c;
```

```
c = a + b;
```

Idea en assembler

```
LOAD R1, a  
LOAD R2, b  
ADD R3, R1, R2  
STORE c, R3
```

- Cargar desde memoria a registros.
- Operar en registros.
- Escribir el resultado de vuelta en memoria.

MIPS

Filosofía MIPS

MIPS (*Microprocessor without Interlocked Pipelined Stages*) es una arquitectura ****RISC**** (Computador de Juego de Instrucciones Reducidas).

Principios de diseño:

- 1 **Simplicidad favorece la regularidad:** Todas las instrucciones miden 32 bits.
- 2 **Lo más pequeño es más rápido:** Solo 32 registros de propósito general.
- 3 **Buen diseño demanda compromisos:** Diferentes formatos pero longitud fija.

Características importantes de MIPS

- Arquitectura **load/store**.
- Banco de **32 registros** de propósito general.
- Instrucciones de 32 bits, con formatos regulares.
- Pocas modalidades de direccionamiento.
- Convención clara para parámetros, retorno y stack.

Consecuencia didáctica

El alumno puede seguir mejor el recorrido de los datos entre memoria, registros y control de flujo.

Conceptos: Assembler vs. MIPS

- **Assembler (Lenguaje Ensamblador):** Es el lenguaje de programación de bajo nivel que utiliza mnemónicos para representar instrucciones binarias.
- **MIPS:** Es una arquitectura de computadoras tipo **RISC** (Reduced Instruction Set Computer) específica.

Conceptos: Assembler vs. MIPS

Característica	Assembler	MIPS
Naturaleza	El lenguaje "humano-legible".	El hardware (ISA).
Dependencia	Cada arquitectura tiene su propio assembler.	Define qué registros y operaciones existen.
Ejemplo	<code>add \$t0, \$s1, \$s2</code>	Registros de 32 bits, sys-calls específicos.
Traducción	El Ensamblador (software) lo pasa a binario.	El procesador ejecuta el binario directamente.

Analogía

Si el **Assembler** es el idioma (español), **MIPS** es el país (España) donde ese idioma tiene reglas y modismos particulares.

El Banco de Registros de MIPS

A diferencia de x86, en MIPS los registros tienen nombres y propósitos específicos por convención:

Registro	Nombre	Uso	Preservado
\$0	\$zero	Constante cero (física)	N/A
\$2-\$3	\$v0-\$v1	Valores de retorno y Syscalls	No
\$4-\$7	\$a0-\$a3	Argumentos de funciones	No
\$8-\$15	\$t0-\$t7	Temporales (uso rápido)	No
\$16-\$23	\$s0-\$s7	Registros salvados	Sí
\$29	\$sp	Stack Pointer (puntero de pila)	Sí
\$31	\$ra	Return Address (dir. de retorno)	No

Instrucciones Frecuentes en MIPS

Instrucción	Ejemplo	Acción
add	add \$t0, \$t1, \$t2	$\$t0 = \$t1 + \$t2$
sub	sub \$s0, \$s1, \$s2	$\$s0 = \$s1 - \$s2$
lw	lw \$t0, 0(\$s1)	Carga de memoria a registro
sw	sw \$t0, 4(\$s1)	Guarda de registro a memoria
beq	beq \$t0, \$t1, Label	Salta si son iguales
li	li \$v0, 4	Carga inmediata (valor fijo)

Formatos de Instrucción

MIPS divide sus 32 bits en 3 estructuras según la necesidad de la operación:

Tipo R

op (6)	rs (5)	rt (5)	rd (5)	sh (5)	funct (6)
--------	--------	--------	--------	--------	-----------

Tipo I

op (6)	rs (5)	rt (5)	immediate (16)
--------	--------	--------	----------------

- **Tipo R:** Aritmética entre registros (add \$rd, \$rs, \$rt).
- **Tipo I:** Cargas/Almacenamiento y Saltos condicionales (lw, sw, beq).

Instrucciones MIPS muy frecuentes

acceso a memoria

lw \$t0, x # load word

sw \$t0, x # store word

aritmetica

add \$t2, \$t0, \$t1

addi \$sp, \$sp, -8

sub \$t3, \$t3, \$t4

comparacion y saltos

beq \$t0, \$t1, etiqueta

bne \$t0, \$t1, otra

j fin

jal funcion

jr \$ra

Ejemplos Básicos: Aritmética y Memoria

Sumar dos números: ($a = b + c$)

lw \$t1, 0(\$s0) # Carga b

lw \$t2, 4(\$s0) # Carga c

add \$t0, \$t1, \$t2 # Suma

sw \$t0, 8(\$s0) # Guarda resultado en a

Multiplicar por 4 usando desplazamiento (SLL)

Un shift a la izquierda de 2 bits equivale a x4

sll \$t0, \$s1, 2

Syscalls y Mensajes

Llamadas al Sistema (Syscall)

Las **syscalls** permiten que el programa pida servicios al Sistema Operativo (o al simulador como MARS/SPIM), como imprimir texto o leer teclado.

Procedimiento Estándar:

- 1 Cargar el **código del servicio** en el registro `$v0`.
- 2 Cargar los **argumentos** (si existen) en los registros `$a0`, `$a1`....
- 3 Ejecutar la instrucción `syscall`.

Ejemplo: *Hola Mundo* en MIPS

hola.asm

```
.data
    mensaje: .asciiz "¡Hola, este es un mensaje!"

.text
main:
    # 1. Imprimir el string
    li $v0, 4      # Código 4 = print_string
    la $a0, mensaje # Cargar dirección del texto
    syscall

    # 2. Terminar el programa
    li $v0, 10     # Código 10 = exit
    syscall
```

Tabla de Syscalls Comunes

\$v0	Servicio	Argumentos
1	Imprimir Entero	\$a0 = valor
4	Imprimir String	\$a0 = dirección
5	Leer Entero	Retorna en \$v0
8	Leer String	\$a0 = buffer, \$a1 = largo
10	Salir (Exit)	Ninguno

Uso de 'li', 'la' y Servicios de Consola

MIPS usa pseudoinstrucciones para facilitar la carga de valores:

- **li** (*Load Immediate*): Carga un número fijo.
- **la** (*Load Address*): Carga una dirección de memoria (puntero).

```
.data
    saludo: .asciiz "Ingrese un valor: "

.text
# 1. Imprimir mensaje (Servicio 4)
li $v0, 4           # Código para print_string
la $a0, saludo       # Dirección del texto
syscall

# 2. Leer entero (Servicio 5)
li $v0, 5           # Código para read_int
syscall             # El valor ingresado queda en $v0
```

Ejemplo Completo: Suma con interacción

```
.data
    res_msg: .asciiz "El resultado es: "
.text
main:
    li $t0, 15      # Cargar primer número
    li $t1, 20      # Cargar segundo número
    add $t2, $t0, $t1 # Sumar

    # Imprimir texto descriptivo
    li $v0, 4
    la $a0, res_msg
    syscall

    # Imprimir el valor de la suma ($t2)
    li $v0, 1
    move $a0, $t2    # Mover $t2 a $a0 para el servicio 1
    syscall

    # Finalizar programa
    li $v0, 10
    syscall
```

Ejemplos varios

Ejemplo 1: Operaciones Aritméticas

Objetivo: Calcular la expresión $Z = (A + B) - (C + 5)$

```
.text
main:
    li $t0, 10      # A = 10
    li $t1, 20      # B = 20
    li $t2, 5        # C = 5

    add $t3, $t0, $t1 # $t3 = A + B (30)
    addi $t4, $t2, 5   # $t4 = C + 5 (10)
    sub $s0, $t3, $t4 # $s0 = $t3 - $t4 (20)

    # Imprimir resultado
    li $v0, 1         # Servicio para imprimir entero
    move $a0, $s0      # Mover resultado a $a0
    syscall
```

Ejemplo 2: Carga y Almacenamiento (Load/Store)

Objetivo: Intercambiar los valores de dos variables en memoria.

```
.data
    var1: .word 100
    var2: .word 200

.text
main:
    la $s0, var1      # Cargar dirección de var1
    la $s1, var2      # Cargar dirección de var2

    lw $t0, 0($s0)     # $t0 = contenido de var1
    lw $t1, 0($s1)     # $t1 = contenido de var2

    sw $t1, 0($s0)     # Guardar valor de var2 en la dir de var1
    sw $t0, 0($s1)     # Guardar valor de var1 en la dir de var2
```

Ejemplo 3: Estructura If-Else

Objetivo: Determinar el mayor de dos números.

```
.text
main:
    li $t0, 50      # x = 50
    li $t1, 80      # y = 80

    # Si (t0 > t1) saltar a EsMayor, sino continuar
    bgt $t0, $t1, EsMayor

    move $a0, $t1    # El mayor es t1 (y)
    j Imprimir

EsMayor:
    move $a0, $t0    # El mayor es t0 (x)

Imprimir:
    li $v0, 1
    syscall
```

Ejemplo 4: Bucle Simple (Contador)

Objetivo: Imprimir los números del 1 al 5.

```
.text
main:
    li $t0, 1      # i = 1
    li $t1, 5      # límite = 5

bucle:
    bgt $t0, $t1, fin # Si i > 5, salir

    # Imprimir contador
    li $v0, 1
    move $a0, $t0
    syscall

    addi $t0, $t0, 1 # i++
    j bucle          # Repetir

fin:
    li $v0, 10      # Exit
    syscall
```

Ventajas y desventajas de MIPS

Ventajas

- Diseño simple y regular.
- Muy útil para aprender ISA, llamadas y stack.
- Separación clara entre memoria y registros.
- Simuladores livianos y fáciles de usar.

Desventajas

- Menos cercana al hardware que hoy domina PCs personales.
- Algunas simplificaciones de los simuladores no reflejan todo un sistema real.
- Para proyectos modernos, hoy suele compararse con ARM o RISC-V.

Herramientas recomendadas: QtSpim y MARS

Herramienta	Fortalezas	Observaciones
QtSpim	simulador MIPS32 clásico, interfaz simple, debugger básico	buena opción para mostrar memoria, registros y syscalls
MARS	IDE educativo en Java, simple de ejecutar, cómodo para prácticas cortas	muy usado en cursos introductorios; es un proyecto legado pero estable

- Si la meta es comprender la ISA: cualquiera de los dos sirve.
- Si se quiere un entorno “portable” por JAR: MARS resulta cómodo.
- Si se prefiere la tradición de SPIM/QtSpim: QtSpim sigue siendo una buena referencia.

MARS 4.5 y MIPS 5.4

Instalación rápida en Windows o Linux

Opción 1: QtSpim

- Windows: descargar el instalador/ZIP y ejecutar QtSpim.
- Linux (Debian/Ubuntu): descargar el paquete .deb de QtSpim e instalarlo.

Opción 2: MARS

- Requiere Java.
- Windows o Linux: descargar el archivo Mars.jar.
- Ejecutar con:

```
java -jar Mars.jar
```

Consejo didáctico: para una primera clase, MARS suele ser la instalación más directa si ya hay JVM disponible.

Programa básico en C

```
int x = 5;
int y = 7;
int z = 0;

int main(void) {
    z = x + y;
    return 0;
}
```

- x, y y z viven en la zona de datos estáticos.
- El trabajo real se hace en registros.
- El resultado final se guarda otra vez en memoria.

Versión equivalente en MIPS

```
.data
x: .word 5
y: .word 7
z: .word 0

.text
.globl main
main:
    lw    $t0, x
    lw    $t1, y
    add   $t2, $t0, $t1
    sw    $t2, z

    li    $v0, 10      # exit
    syscall
```

Lectura del ejemplo MIPS

- ❶ `lw $t0, x`: carga `x` desde memoria al registro `$t0`.
- ❷ `lw $t1, y`: carga `y` en `$t1`.
- ❸ `add $t2, $t0, $t1`: suma en registros.
- ❹ `sw $t2, z`: guarda el resultado en memoria.
- ❺ `syscall`: termina el programa en el simulador.

Idea clave

En MIPS, la ALU trabaja con registros; la memoria se toca explícitamente con `lw/sw`.

Cómo trabajar con un programa básico en C

Flujo didáctico simple

- 1 Escribir un programa muy pequeño en C.
- 2 Identificar variables, operaciones y control.
- 3 Traducir manualmente a MIPS.
- 4 Ejecutar en QtSpim o MARS.
- 5 Observar registros y memoria paso a paso.

Flujo con toolchain (Linux)

```
sudo apt install gcc-mips-linux-gnu  
mips-linux-gnu-gcc -S -O0 suma.c -o  
↪ suma.s
```

Esto permite inspeccionar assembly generado por compilador, aunque para una primera clase suele ser más claro empezar con traducción manual.

Resumen final

- Assembler expone cómo la CPU manipula registros, memoria y saltos.
- MIPS es una ISA muy útil para aprender el modelo **load/store** y las convenciones de llamada.
- Un programa sencillo en C puede desglosarse en cargas, operaciones en registros y escrituras a memoria.
- QtSpim y MARS permiten experimentar paso a paso sin depender del hardware real.

Recursos sugeridos

- Sitio oficial de SPIM / QtSpim.
- Sitio oficial / repositorio de MARS.
- Patterson & Hennessy, *Computer Organization and Design*.
- Práctica recomendada: ejecutar el ejemplo, cambiar constantes y luego agregar un `if` o una función sencilla.

Arbol de activación

Idea central

- La **jerarquía de bloques** es **estática**: se obtiene leyendo el programa.
- El **árbol de activación** es **dinámico**: se construye durante la ejecución.
- Cada llamada a función crea una **activación** y, por lo general, un **registro de activación** (stack frame).
- En C, los bloques internos crean **alcances** para variables locales, pero normalmente **no** crean un nuevo stack frame.

Pregunta guía

¿Cómo se relaciona la estructura del código con la forma en que las activaciones aparecen en memoria?

Ejemplo en C

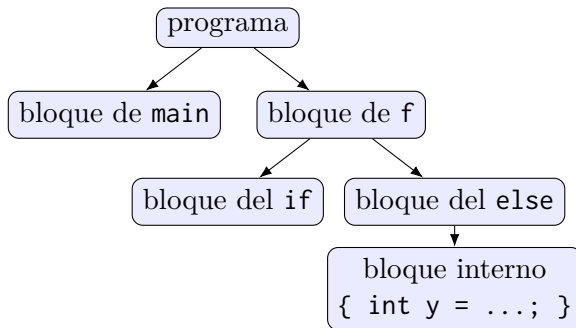
```
int f(int n)
{
    int x = n;

    if (n <= 1) {
        int base = 1;
        return base;
    } else {
        int parcial;

        {
            int y = n - 1;
            parcial = f(y);
        }

        return x + parcial;
    }
}
```

Jerarquía de bloques (estructura estática)



- Esta jerarquía existe **antes** de ejecutar el programa.
- Describe **alcances léxicos** y dónde son visibles los identificadores.

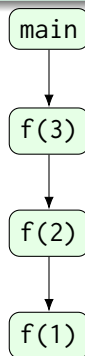
Traza de ejecución para main $\rightarrow$ f(3)

- 1 main llama a f(3)
- 2 f(3) llama a f(2)
- 3 f(2) llama a f(1)
- 4 f(1) retorna 1
- 5 f(2) retorna $2 + 1 = 3$
- 6 f(3) retorna $3 + 3 = 6$
- 7 main finaliza

Observación

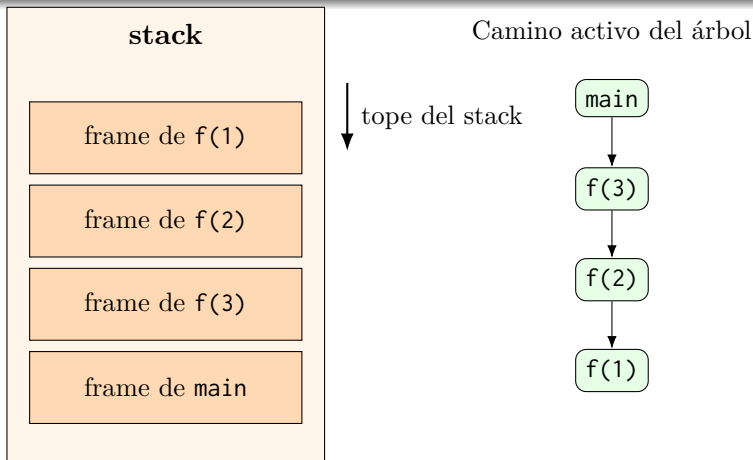
La función f aparece **varias veces** en ejecución, aunque en el código fuente haya **un solo bloque** para f.

Árbol de activación (ejecución dinámica)



- Cada nodo representa **una activación concreta**.
- Si la función se llama recursivamente, aparecen **múltiples nodos** para la misma función.
- El árbol depende de los datos de entrada y del flujo real de ejecución.

Del árbol al stack



Propiedad importante: los frames presentes en el stack corresponden al **camino activo** desde la raíz del árbol de activación hasta la activación actual.

Contenido de una activación de f

frame de $f(n)$

parámetro n

dirección de retorno

local x

local parcial

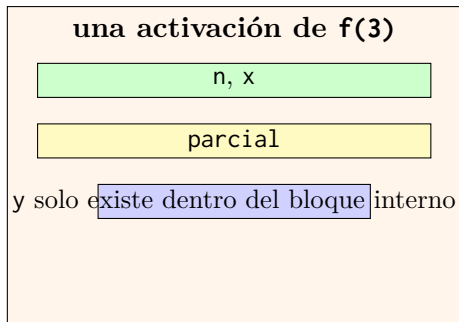
espacio variables bloque

temporales / registros

En C:

- base y y tienen alcance de bloque.
- Memoria resuelta **dentro del mismo frame.**
- El compilador puede **reutilizar** espacio.

Bloques internos y tiempo de vida



Conclusión

- La jerarquía de bloques determina **visibilidad**.
- El árbol de activación determina **cuántas activaciones** existen al mismo tiempo.
- El stack refleja solo las **activaciones activas**, no todos los bloques del programa.

Comparación directa

Jerarquía de bloques

- Estática
- Surge del texto fuente
- Describe alcance léxico
- Una sola por programa

Árbol de activación

- Dinámico
- Surge durante la ejecución
- Describe llamadas y retornos
- Puede cambiar según la entrada

En C

Los bloques anidados **no implican** nuevos frames. Los frames se crean por **llamadas a funciones**.

Lectura guiada del ejemplo

```
r = f(3);  
// activaciones: main -> f(3)  
  
parcial = f(y); // en f(3), y vale 2  
// activaciones: main -> f(3) -> f(2)  
  
parcial = f(y); // en f(2), y vale 1  
// activaciones: main -> f(3) -> f(2) -> f(1)  
  
return base; // f(1) retorna 1  
return x + parcial; // f(2) retorna 3  
return x + parcial; // f(3) retorna 6
```

- Cada **return** elimina el frame superior.
- El stack se desapila en orden **LIFO**.

Resumen

- **Jerarquía de bloques:** organización léxica del programa.
- **Árbol de activación:** historia de llamadas de una ejecución concreta.
- **Stack:** contiene los registros de activación del camino activo.
- En C, los bloques internos afectan el **alcance** y el **tiempo de vida** de ciertas variables, pero no suelen crear un frame propio.

Idea para recordar

Bloques → visibilidad

| Activaciones → ejecución y memoria

Class Instance Record - CIR

Idea general

Definición intuitiva

El **CIR** es la **estructura de memoria de una instancia de clase**, es decir, el bloque que representa a un objeto concreto en tiempo de ejecución.

Incluye típicamente:

- referencia al tipo o metadatos del objeto,
- acceso a la **vtable** o tabla de métodos virtuales,
- campos heredados,
- campos propios de la clase.

¿En qué lenguajes aparece esta idea?

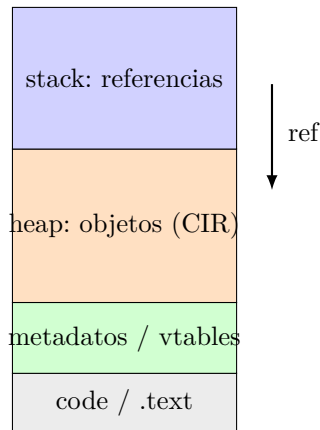
- **C++**: layout de objetos con puntero implícito a vtable en casos polimórficos.
- **Java**: cada objeto tiene cabecera + campos; conceptualmente se usa una tabla de despacho por clase.
- **C#**: la VM mantiene metadatos de tipo y despacho dinámico.
- **Smalltalk / Objective-C / Ruby**: fuerte dependencia del objeto y del envío de mensajes.

Idea didáctica

Aunque cada runtime implementa detalles distintos, el concepto pedagógico es el mismo: **una instancia necesita una representación concreta en memoria.**

Relación con la memoria

- La **referencia al objeto** suele vivir en el **stack**.
- El **CIR del objeto** vive en el **heap**.
- Los métodos están en **code/.text**.
- La vtable se asocia a la **clase**, no a cada método por separado.



Estructura conceptual de un CIR

header: tipo / GC / lock / metadatos
referencia a vtable / descriptor de clase
campos heredados
campos propios
alineación / padding / referencias a otros objetos

Ejemplo base en Java

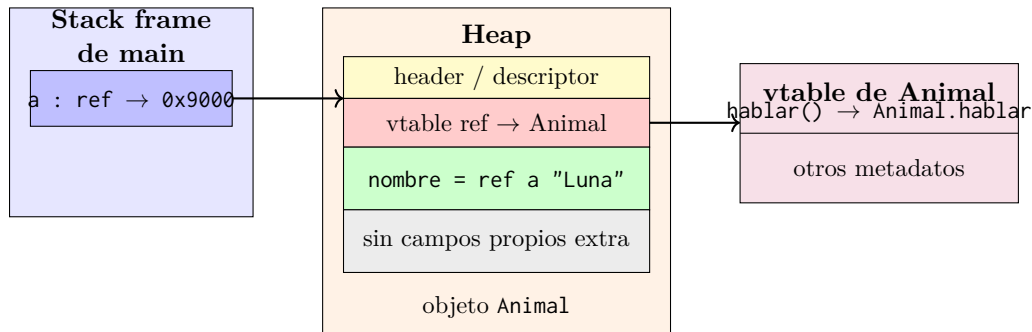
```
1  class Animal {
2      String nombre;
3
4      Animal(String n) { nombre = n; }
5
6      void hablar() {
7          System.out.println("sonido generico");
8      }
9  }
10
11 class Perro extends Animal {
12     int edad;
13
14     Perro(String n, int e) {
15         super(n);
16         edad = e;
17     }
18
19     @Override
20     void hablar() {
21         System.out.println("guau");
22     }
```

Caso 1: referencia y objeto de la misma clase

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal("Luna");  
        a.hablar();  
    }  
}
```

- El tipo estático de `a` es `Animal`.
- El tipo dinámico del objeto también es `Animal`.
- No hay cambio de clase entre referencia y objeto.
- El CIR del heap corresponde a un objeto `Animal`.

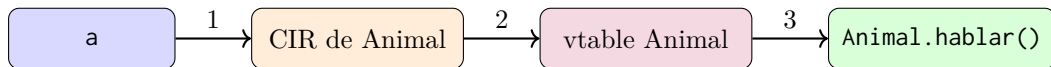
CIR y memoria



Idea

La referencia vive en el stack, el CIR vive en el heap y la vtable usada es la de **Animal**.

Despacho del método



- ❶ Se lee la referencia `a`.
- ❷ Se accede al CIR del objeto.
- ❸ Se sigue la referencia a la vtable de `Animal`.
- ❹ La entrada `hablar()` apunta a `Animal.hablar()`.

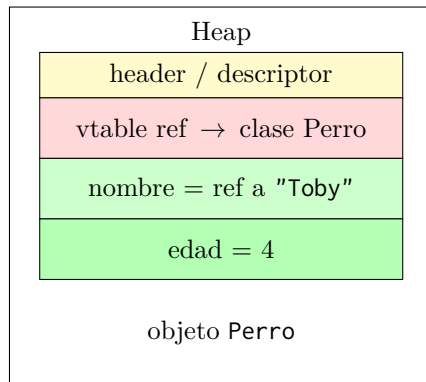
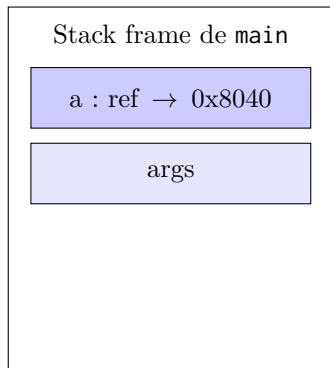
Caso 2: ejemplo base ideal

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Animal a = new Perro("Toby", 4);  
4         a.hablar();  
5     }  
6 }
```

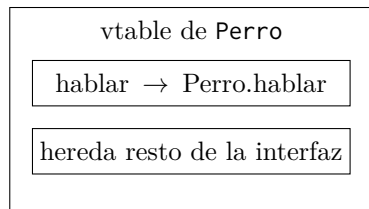
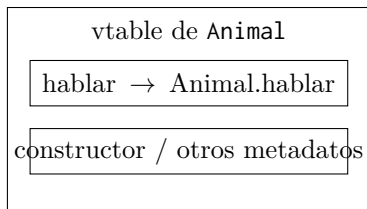
Preguntas clave:

- ¿Dónde está a?
- ¿Dónde está el objeto new Perro("Toby", 4)?
- ¿Cómo sabe la ejecución que debe llamar a Perro.hablar()?

Estado de memoria durante la ejecución



Una vtable por clase



Clave

El objeto **no contiene el código del método**. Contiene una referencia que permite llegar a la tabla de métodos correspondiente a su clase.

Despacho dinámico paso a paso

Para ejecutar `a.hablar()`:

- 1 Se lee la referencia `a` desde el stack.
- 2 Se accede al CIR del objeto en el heap.
- 3 Se sigue la referencia a la **vtable de Perro**.
- 4 Se busca la entrada correspondiente a `hablar()`.
- 5 Se salta al código de `Perro.hablar()` en el segmento de código.



Caso 3: Perro p = new Animal()

```
public class Main {  
    public static void main(String[] args) {  
        Perro p = new Animal("Luna");    // error  
    }  
}
```

Caso 3: Perro p = new Animal()

```
public class Main {  
    public static void main(String[] args) {  
        Perro p = new Animal("Luna");    // error  
    }  
}
```

Qué ocurre

Este código **no compila**.

Caso 3: Perro p = new Animal()

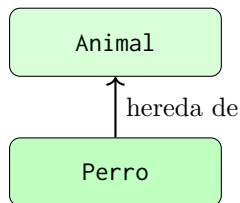
```
public class Main {  
    public static void main(String[] args) {  
        Perro p = new Animal("Luna");    // error  
    }  
}
```

Qué ocurre

Este código **no compila**.

- p tiene tipo estático Perro.
- new Animal("Luna") crea un objeto cuyo tipo dinámico es Animal.
- Un Animal no puede asignarse directamente a una referencia de tipo Perro.
- El chequeo ocurre en compilación.

Caso 3: por qué el compilador lo rechaza



Regla de tipos

Es válido asignar una subclase a una referencia de superclase:

```
Animal a = new Perro(...);
```

No es válido asignar una superclase a una referencia de subclase:

```
Perro p = new Animal(...);
```

Consecuencia

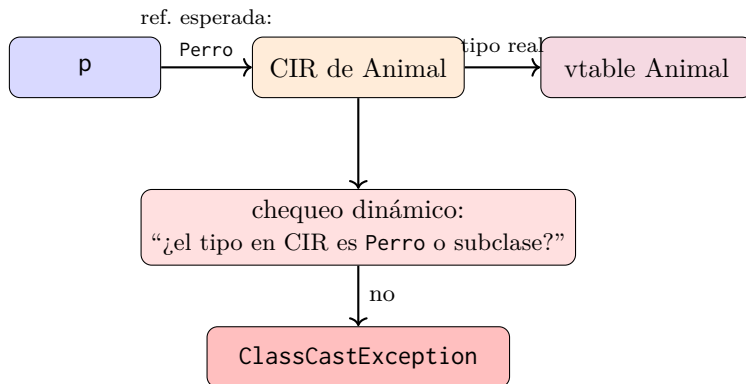
Como el programa no pasa el chequeo estático, no llegamos a tener una asignación válida entre esa referencia y ese CIR.

Variante útil: cast explícito

```
public class Main {  
    public static void main(String[] args) {  
        Perro p = (Perro) new Animal("Luna");  
        p.hablar();  
    }  
}
```

- Ahora el código **sí compila**, porque hay un cast explícito.
- Pero el objeto creado sigue siendo dinámicamente un `Animal`.
- En ejecución, la VM verifica si ese objeto realmente es un `Perro`.
- Como no lo es, se produce `ClassCastException`.

Cast explícito: chequeo dinámico



Idea clave

El cast es una operación de ****lectura y comparación****. El objeto en memoria no se modifica; el sistema simplemente inspecciona el descriptor de clase (CIR) para confirmar si la operación es segura.

Comparación de los tres escenarios

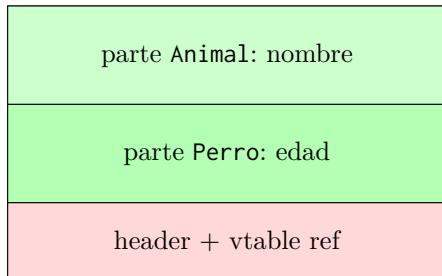
Código	Compila	Ejecuta	CIR / vtable usada
<code>Animal a = new Animal(...);</code>	Sí	Sí	CIR de Animal, vtable Animal
<code>Animal a = new Perro(...);</code>	Sí	Sí	CIR de Perro, vtable Perro
<code>Perro p = new Animal(...);</code>	No	No llega	asignación inválida por tipos
<code>Perro p = (Perro) new Animal(...);</code>	Sí	No	CIR de Animal, chequeo dinámico falla

Mensaje final

La referencia puede cambiar, pero el CIR y la vtable dependen del **tipo real del objeto creado en el heap**.

Relación con la herencia

- El CIR de una subclase suele incluir primero la parte correspondiente a la superclase.
- Luego agrega sus propios campos.
- Esto facilita que un objeto de subclase pueda ser tratado como uno de la superclase.



Diferencia entre CIR y registro de activación

Registro de activación	Class Instance Record (CIR)
Representa una llamada a función o método .	Representa una instancia de clase .
Suele estar en el stack .	Suele estar en el heap .
Contiene parámetros, retorno, variables locales.	Contiene campos del objeto, header y acceso a vtable.
Vive mientras la llamada está activa.	Vive mientras el objeto siga alcanzable.

Ventajas didácticas del modelo

- Permite explicar **objetos como estructuras de memoria reales**.
- Conecta **herencia con layout en memoria**.
- Permite introducir **despacho dinámico, vtables y polimorfismo**.
- Ayuda a diferenciar claramente:
referencia, objeto, clase y código del método.

Limitaciones y aclaraciones

- **CIR** es un concepto didáctico frecuente; no todos los lenguajes usan exactamente ese nombre.
- El layout exacto depende del **runtime**, la **VM**, el **compilador** y la **arquitectura**.
- En Java real hay detalles extra: cabeceras, compresión de referencias, alineación, GC, etc.
- En C++ el modelo suele verse más directamente cuando se estudian **objetos polimórficos y vtables**.

Resumen

Idea central

Un **Class Instance Record** es la representación en memoria de un objeto.

- La referencia al objeto suele estar en el stack.
- El objeto vive en el heap.
- El CIR contiene campos y acceso al comportamiento dinámico.
- La vtable pertenece a la clase y permite resolver métodos en ejecución.

Referencias

-  AHO, ALFRED, SETHI, RAVI, ULLMAN, JEFFREY D., LAM, MONICA S., 2008, *Compiladores. Principios, técnicas y herramientas*, Addison Wesley.
-  SEBESTA, ROBERT W., 2016, *Concepts of Programming Languages*, Pearson.
-  SCOTT, MICHAEL L., 2015, *Programming Language Pragmatics*, Morgan Kaufmann.
-  PATTERSON, DAVID A., HENNESSY, JOHN L., 2014, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, Morgan Kaufmann.